Informe Técnico: Arquitectura Backend, Integración de Machine Learning y Servidor FastAPI

Proyecto: SportMind ML API

Objetivo: Proveer un servicio backend robusto y de alto rendimiento para la predicción de la autorregulación emocional en atletas mediante la integración de un modelo predictivo entrenado con scikit-learn.

1. Descripción del Modelo de Machine Learning y su Propósito

El núcleo analítico de la API se encarga de evaluar el estado psicológico y de rendimiento de un atleta para clasificar su nivel de **autorregulación emocional**.

- **Funcionalidad del modelo:** A partir de un conjunto multidimensional de variables (demográficas, emocionales previas y métricas de desempeño físico), el modelo predice si el atleta presenta una *Autorregulación Óptima* (etiqueta 1) o si *Requiere Intervención / Desregulado* (etiqueta 0).
-
- **Variables de Entrada (Features):** El modelo procesa un vector de 20 características que incluye:
-
    - **Datos demográficos y deportivos:** Deporte seleccionado, género y estado emocional inicial.
    -
    - **Pruebas de precisión (Tiro):** Niveles emocionales previos y posteriores, puntajes en modalidades fácil y difícil, rendimiento, ritmo y confianza (escala de 1 a 5).
    -
    - **Pruebas físicas (Escalada/Muro):** Tiempos de ejecución en rutas fáciles y difíciles, rendimiento, ritmo, confianza y recomendación final de desempeño.
    -
- **Salidas (Outputs):** El sistema retorna la clase predicha, una etiqueta descriptiva legible, la probabilidad asociada de certeza y la lista de características utilizadas.
-

2. Arquitectura del Backend con FastAPI

Para exponer el modelo de forma eficiente y segura, se desarrolló una API utilizando **FastAPI**, aprovechando su validación automática y documentación interactiva nativa.

- **Esquemas de Validación (Pydantic):** Se implementó la clase `DatosAtleta`, la cual define y valida tipados estrictos (`str`, `float`, `int`) y valores por defecto seguros para

cada uno de los parámetros requeridos por el modelo, previniendo fallos ante solicitudes mal formadas.

- **Definición de Endpoints:**
    - `GET /`: Endpoint de diagnóstico y salud (health check) que confirma si el servicio está en línea, valida el estado de carga del modelo e inspecciona los metadatos y características esperadas.
    - `POST /predict`: Endpoint principal que recibe los datos del atleta en formato JSON, ejecuta el pipeline de preprocesamiento y realiza la inferencia en tiempo real.

3. Integración, Carga y Transformación de Datos

El script de la API (`api.py`) maneja la deserialización del modelo y la normalización de los datos de entrada mediante un flujo estructurado:

- **Carga del Bundle (`joblib`):** Al arrancar el servidor, se carga de forma segura el archivo `modelo_autoregulacion_emocional.pkl`, extrayendo dinámicamente el estimador de Machine Learning, el objeto de normalización (`scaler`), los codificadores categóricos (`encoders`) y los metadatos de entrenamiento.
- **Codificación y Sanitización Segura:** Se implementaron funciones auxiliares (`safe_encode` y `safe_float`) para traducir de manera robusta los valores de texto categóricos a índices numéricos mediante los `LabelEncoder` originales, y para asegurar la conversión correcta de los datos numéricos evitando errores por nulos o tipos de datos incorrectos.
- **Vectorización y Escalado:** Los valores procesados se agrupan en un vector bidimensional de 20 características (`numpy.ndarray`), el cual es transformado por el `scaler` del modelo antes de ejecutar la predicción mediante `modelo.predict()` y `modelo.predict_proba()`.

4. Configuración de Seguridad y Servidor ASGI (Uvicorn)

- **Gestión de CORS:** Se configuró `CORSMiddleware` para permitir solicitudes seguras desde entornos de desarrollo frontend (como dashboards en Astro sobre `localhost:4321`) y clientes externos de prueba.
- **Despliegue y Ejecución:** La aplicación se ejecuta mediante el servidor ASGI **Uvicorn**, operando de manera local con recarga automática para pruebas de desarrollo (`reload=True`), y desplegada exitosamente en la infraestructura en la nube de Render, donde el sistema valida de forma automática el arranque del servidor y la correcta inicialización del modelo predictivo.